COAL

Lecture 10

Video Memory

- **Definition**: Video memory (VRAM) is a portion of system memory used to store data for display.
- **Purpose**: Holds characters, pixels, and attributes that define how the screen content appears.
- **Location**:
  - Text mode video memory starts at physical address 0xB8000
  - Commonly used in assembly for direct screen manipulation.

Text Mode Video Memory Layout

- **Memory Mapping**:
  - Example for an 80x25 screen resolution:
  - There are 25 rows, and each row has 80 characters
    - Total cells = 80 columns × 25 rows = 2000 cells
    - Each cell = 2 bytes, so total memory = 4000 bytes
- **Structure**:
  - Each character cell consists of two bytes:
    - **Byte 1**: ASCII code for the character
    - **Byte 2**: Attribute byte (defines colors and effects)

Attribute Byte Details

- **Structure**:
  - 4 least significant bits (first nibble) for foreground color.
  - 4 most significant bits (second nibble) for background color.
- Each character's background and foreground colors can be set or obtained by reading a specific byte from memory.
- RGB are red, green, and blue colors. ‘I’ is the intensity of the color, and ‘B

Background Color				Foreground Color			
B	R	G	B	I	R	G	B
Blink	Red	Green	Blue	Intensity	Red	Green	Blue

Attribute Color Codes

	I/B	R	G	B	Color
0x0	0	0	0	0	Black
0x1	0	0	0	1	Blue
0x2	0	0	1	0	Green
0x3	0	0	1	1	Cyan
0x4	0	1	0	0	Red
0x5	0	1	0	1	Magenta
0x6	0	1	1	0	Brown
0x7	0	1	1	1	White
0x8	1	0	0	0	Gray
0x9	1	0	0	1	Light Blue
0xA	1	0	1	0	Light Green
0xB	1	0	1	1	Light Cyan
0xC	1	1	0	0	Light Red
0xD	1	1	0	1	Light Magenta
0xE	1	1	1	0	Yellow
0xF	1	1	1	1	Bright

Example:

- `0x1E` :
  - Foreground: Light Yellow (0xE).
  - Background: Blue (0x1).

Direct Accessing Video Memory

- **Register Setup**:
  - ES: Segment register pointing to video memory (usually 0xB8000 in text mode)
  - SI/DI: Offset within the segment for a specific character cell.
- **Basic Steps**:
  - Load video memory segment into ES
  - Calculate offset for desired screen position
  - Write ASCII and attribute bytes

Common ASCII Codes

Character	ASCII (Decimal)	ASCII (Hex)
A	65	0x41
a	97	0x61
0	48	0x30
Space	32	0x20
Enter	13	0x0D

Example

```
.model small
.stack 100h
.data
.code
Mov ax,0xb800
Mov es, ax
Mov si,0
Mov es:[si], 'H'
Inc si
Mov byte ptr es:[si], 11100100b
Inc si
Mov es:[si], 'E'
Inc si
Mov byte ptr es:[si], 11100100b
Inc si
Mov es:[si], 'L'
Inc si
Mov byte ptr es:[si], 11100100b
Inc si
Mov es:[si], 'L'
Inc si
Mov byte ptr es:[si], 11100100b
Inc si
Mov es:[si], 'O'
Inc si
Mov byte ptr es:[si], 11100100b
Inc si
.exit
```

Screen Coordinates

- The screen is two-dimensional, having rows and columns,
- While the memory containing the screen's contents is linear.
- Each character on a screen has a row and column address,
- Which is represented as (row, column)
- It can be translated to a linear memory address by using the following equations:

Equations

- Equation 1 finds the linear address of the memory location where the ASCII code of the character to be shown on screen will be stored.
  - Linear Address (ASCII) = 2 * (Row address * Columns per Row + Column address)
- Equation 2 finds the linear address of the memory location where the character's screen attributes will be stored.
  - Linear Address (Attributes) = 2 * (Row address * Columns per Row + Column address) + 1
- This linear address will be added to the base address, i.e., 0xB8000, to form the physical address.
- Here, Number of rows = 25; Columns per Row = 80

Example

- To display a text on the middle of the screen, we need to know its screen coordinates which are (12,40).
- These coordinates can be translated into physical address by using equation 1 as shown below.
  - Memory Address (ASCII) = 2 * (12 * 80 + 40) = 2000
- Similarly, the offset address of the attributes of center character is:
  - Memory Address (Attribute) = 2 * (12 * 80 + 40) +1 = 2000 + 1 = 2001

Example

```
.model small
.stack 100h
.data
.code
Mov ax,0xb800
Mov es,ax
Mov si,2000
Mov es:[si], 'H'
add si,2
Mov es:[si], 'E'
add si,2
Mov es:[si], 'L'
add si,2
Mov es:[si], 'L'
add si,2
Mov es:[si], 'O'
.exit
```

Defining Strings

- A sequence of characters stored in consecutive memory locations.
- Strings are defined using a byte array as shown below
  - Msg db “Hello World”
- A string is terminated by a character which can be NULL or ‘$’ so that the procedure handling string could know that the string has been ended.
- We will place 0, i.e., NULL character at the end of a string as shown below
  - Msg db “Hello World”,0

Example – with Jump

```
.model small
.stack 100h
.data
msg db "Hello World",0
.code
mov ax,@data
mov ds,ax
Mov ax,0xB800
Mov es,ax
Mov di,0
mov bx,offset msg
mov si,0
display:
mov al, [bx+si]
; mov one character from array to al
cmp al,0
; compare is that character is null
je end ; if null, terminate loop
mov es:[di],al
; Otherwise, mov that character to video memory
add di,2
; add 2 to di, skipping the attribute part
inc si
; inc si to access the next character of array
jmp display ; iterate loop
end:
.exit
```

Example – with Loop

```
.model small
.stack 100h
.data
msg db "Hello World"
len equ $- msg
.code
mov ax,@data
mov ds,ax
mov cx,len
Mov ax,0xB800
Mov es,ax
Mov di,0
LEA BX, msg
; LEA stands for Load Effective Address
; same effect like this instruction (mov bx, offset msg)
mov si,0
display:
mov al, [bx+si]
; mov one character from array to al
mov es:[di],al
; mov that character to video memory
add di,2
; add 2 to di, skipping the attribute part
inc si
; inc si to access the next character of array
loop display ; iterate loop
end:
.exit
```

Combining Color Code, Screen Attributes and String Display

```
.model small
.stack 100h
.data
msg db "Hello World",0
.code
mov ax,@data
mov ds,ax
Mov ax,0xB800
Mov es,ax
Mov di,2000
mov bx,offset msg
mov si,0
display:
mov al, [bx+si] ; mov one character from array to al
cmp al,0        ; compare is that character is null
je end          ; if null, terminate loop
mov es:[di],al  ; Otherwise, mov that character to video memory
inc di
Mov byte ptr es:[di], 11100100b ; setting the attribute part
inc di
inc si                          ; inc si to access the next character of array
jmp display                     ; iterate loop
end:
.exit
```

Practice Problem

- Write an assembly program to display your name at the center of the screen in red text with a white background.

String Instructions

Instruction	Operands	Operation

LODSB
 	
Also see: LODSW
	No operands	• **AL = DS:[SI]**
		• if DF = 0 then
		  ○ SI = SI + 1
		else
		  ○ SI = SI – 1

STOSB
 	
Also See: STOSW
	No operands	• **ES:[DI] = AL**
		• if DF = 0 then
		  ○ DI = DI + 1
		else
		  ○ DI = DI – 1

MOVSB
 	
Also See: MOVSW
	No operands	• **ES:[DI] = DS:[SI]**
		• if DF = 0 then
		  ○ SI = SI + 1
		  ○ DI = DI + 1
		else
		  ○ SI = SI - 1
		  ○ DI = DI – 1

SCASB
 	
Also See: SCASW
	No operands	• **AL - ES:[DI]**
		• set flags according to result:
		OF, SF, ZF, AF, PF, CF
		• if DF = 0 then
		  ○ DI = DI + 1
		else
		  ○ DI = DI - 1

Example

```
.model small
.stack 100h
.data
msg1 db "Hello World$",0   ; Source string with null terminator
msg2 db 12 dup(?)          ; Destination buffer
.code
main proc
    mov ax, @data          ; Initialize data segment
    mov ds, ax
    mov es, ax

    ; Copy msg1 to msg2
    mov si, offset msg1     ; Offset of msg1 to SI
    mov di, offset msg2     ; Offset of msg2 to DI
    mov cx, 12              ; Length of the string to copy
copy_loop:
    movsb                   ; Copy byte from [SI] to [DI]
    loop copy_loop          ; Decrement CX and repeat until CX = 0

    ; Display msg2 using DOS interrupt
    mov dx, offset msg2     ; Offset of msg2 to DX
    mov ah, 09h             ; Function 09h: Display string
    int 21h                 ; Call DOS interrupt

    ; Exit program
    mov ah, 4Ch             ; Function 4Ch: Terminate program
    int 21h
main endp
end main
```